INSTITUTO FEDERAL DO RIO GRANDE DO NORTE – IFRN
Campus Currais Novos

TÉCNICAS DE ANÁLISE DE DADOS

Atividade 4 – Modelos de classificação
Naive Bayes e Árvores de Decisão

Professor: Marcílio Meira
Alunos: Robério Batista de Azevêdo Júnior e Daniel Goes

Conjunto de dados: Medical Cost Personal Datasets (insurance.csv)

Currais Novos – RN

1. Objetivo

Esta atividade apresenta a aplicação de dois algoritmos de classificação, Naive Bayes e Árvore de Decisão, ao conjunto de dados de custos médicos (`insurance.csv`). Os modelos são avaliados por validação cruzada, e os hiperparâmetros da Árvore de Decisão são selecionados com `GridSearchCV`.

Esclarecimento: `charges` é uma variável numérica contínua. Para viabilizar a classificação e manter o foco do projeto, foram criadas três categorias de custo: Baixo, Médio e Alto. Essas categorias não existem originalmente no arquivo.

2. Preparação dos Dados

A base é carregada da pasta `dados/` e passa por verificação de registros duplicados e valores ausentes, considerando a EDA realizada anteriormente. Os dados são separados em treinamento (80%) e teste (20%). As faixas de custo são definidas com `pd.qcut()` exclusivamente no conjunto de treinamento; os mesmos limites são utilizados no conjunto de teste.

A variável `charges` não é utilizada como atributo preditor. Para evitar vazamento de informações, a padronização numérica e a codificação das categorias são aplicadas por um `Pipeline` dentro de cada divisão da validação cruzada. O pré-processamento da Atividade 2 permanece documentado na entrega anterior; nesta atividade ele é integrado aos classificadores para possibilitar a validação correta.

3. Modelos de Classificação

Modelo	Implementação	Finalidade
Naive Bayes	GaussianNB	Classificar os registros com base em probabilidades.
Árvore de Decisão	DecisionTreeClassifier	Classificar com regras de decisão sobre os atributos.

4. Otimização e Validação Cruzada

Para a Árvore de Decisão, `GridSearchCV` testa combinações de critério de divisão, profundidade máxima e quantidade mínima de amostras nas folhas, selecionando o melhor conjunto de hiperparâmetros com F1-macro. Em ambos os classificadores, `cross_val_score` utiliza cinco divisões para calcular a acurácia. A Árvore utiliza validação cruzada aninhada, na qual a busca de hiperparâmetros é realizada em cada dobra externa.

A avaliação complementar contempla acurácia, F1-macro, relatório de classificação e matriz de confusão no conjunto de teste reservado. O resumo dos resultados foi salvo em `resultados/atividade4_comparativo.csv`.

5. Resultados Obtidos

Os dois modelos foram executados com o arquivo original `insurance.csv`, contendo 1.338 registros. Após a retirada de uma linha repetida, foram separados 1.069 registros para treinamento e 268 para teste. As faixas foram estabelecidas com os limites de aproximadamente US$ 6.389,38 e US$ 13.012,21, calculados no treinamento.

Métrica	Naive Bayes	Árvore de Decisão
Acurácia média (validação cruzada)	59,68%	86,25%
Desvio padrão (validação cruzada)	10,66 p.p.	1,22 p.p.
Acurácia no teste	57,84%	91,04%
F1-macro no teste	0,4887	0,9096

Na validação cruzada com cinco divisões, a Árvore de Decisão obteve maior acurácia média e menor variação entre as dobras. No conjunto de teste, acertou 244 de 268 registros, contra 155 de 268 do Naive Bayes.

6. Matrizes de Confusão

Naive Bayes — rótulos reais nas linhas, previsões nas colunas

Classe real	Prev. Baixo	Prev. Médio	Prev. Alto
Baixo (100)	100	0	0
Médio (90)	90	0	0
Alto (78)	23	0	55

Árvore de Decisão — rótulos reais nas linhas, previsões nas colunas

Classe real	Prev. Baixo	Prev. Médio	Prev. Alto
Baixo (100)	93	7	0
Médio (90)	3	87	0
Alto (78)	1	13	64

O Naive Bayes não identificou nenhum dos 90 registros da classe Médio (recall = 0). A Árvore de Decisão identificou corretamente 87 desses 90 registros (recall próximo de 97%).

7. Melhores Hiperparâmetros e Considerações Finais

A busca obrigatória com `GridSearchCV` selecionou, para a Árvore de Decisão:

Hiperparâmetro	Melhor configuração
criterion	entropy
max_depth	5
min_samples_leaf	1

A Árvore de Decisão superou o Naive Bayes na validação cruzada e no conjunto de teste, sobretudo na classe Médio, que o Naive Bayes não identificou. Essa vantagem foi observada somente neste experimento e não demonstra superioridade geral do algoritmo.

8. Código-fonte Utilizado

O código completo a seguir corresponde ao arquivo `codigo/atividade_04_classificacao.py`. Para executá-lo na raiz do repositório, use: `python3 codigo/atividade_04_classificacao.py`. Bibliotecas: `pandas`, `numpy` e `scikit-learn`. O pré-processamento da Atividade 2 continua documentado na entrega anterior.

```
"""Atividade 4 - Classificacao com Naive Bayes e Arvore de Decisao.

IFRN - Tecnicas de Analise de Dados
Dupla: Roberio Batista de Azevedo Junior e Daniel Goes
Dados: Medical Cost Personal Datasets (dados/insurance.csv)

A coluna charges e continua. Para manter o foco das atividades anteriores,
criamos as classes Baixo, Medio e Alto usando os tercis de charges no TREINO.
Essas classes nao existem originalmente no arquivo insurance.csv.
"""

from pathlib import Path

import numpy as np
import pandas as pd
from sklearn.compose import ColumnTransformer
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix, f1_score
from sklearn.model_selection import (
    GridSearchCV, StratifiedKFold, cross_val_score, train_test_split,
)
from sklearn.naive_bayes import GaussianNB
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.tree import DecisionTreeClassifier


RAIZ = Path(__file__).resolve().parents[1]
ARQUIVO = RAIZ / "dados" / "insurance.csv"
RESULTADOS = RAIZ / "resultados"
CLASSES = ["Baixo", "Medio", "Alto"]


def carregar_e_separar():
    """Limpeza guiada pela EDA e classes definidas pelo treino."""
    df = pd.read_csv(ARQUIVO)
    print("Dimensoes originais:", df.shape)
    print("Valores ausentes:", int(df.isna().sum().sum()))
    print("Duplicatas:", int(df.duplicated().sum()))
    df = df.drop_duplicates().copy()
    if df.isna().any().any():
        raise ValueError("A base possui valores ausentes nao tratados.")

    # A coluna-alvo continua nao pode aparecer entre os preditores.
    X = df.drop(columns=["charges"])
    custos = df["charges"]
    X_treino, X_teste, custos_treino, custos_teste = train_test_split(
        X, custos, test_size=0.20, random_state=42, stratify=X["smoker"]
    )

    # Mesma proposta da Atividade 3: tres faixas de custo por quantis.
    # Os limites sao calculados SOMENTE no conjunto de treinamento.
    y_treino, limites = pd.qcut(
        custos_treino, q=3, labels=CLASSES, retbins=True
    )
    limites[0], limites[-1] = -np.inf, np.inf
    y_teste = pd.cut(
        custos_teste, bins=limites, labels=CLASSES, include_lowest=True
    )
    print("Registros apos limpeza:", len(df))
    print("Treino e teste:", len(X_treino), len(X_teste))
    print("Distribuicao dos rotulos de treino:\n", y_treino.value_counts())
    print("Limites internos (US$):", [round(x, 2) for x in limites[1:-1]])
    return X_treino, X_teste, y_treino, y_teste


def criar_pipeline(classificador):
    """Prepara as variaveis DENTRO de cada dobra da validacao cruzada."""
    pre = ColumnTransformer(
        transformers=[
            ("numericas", StandardScaler(), ["age", "bmi", "children"]),
            ("categoricas", OneHotEncoder(handle_unknown="ignore"),
             ["sex", "smoker", "region"]),
        ],
        sparse_threshold=0.0,  # GaussianNB exige matriz densa.
```

```
    )
    return Pipeline([("preprocessamento", pre), ("classificador", classificador)])


def main():
    X_treino, X_teste, y_treino, y_teste = carregar_e_separar()
    cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

    # 1. Naive Bayes Gaussiano + validacao cruzada OBRIGATORIA.
    naive_bayes = criar_pipeline(GaussianNB())
    scores_nb = cross_val_score(
        naive_bayes, X_treino, y_treino, cv=cv, scoring="accuracy"
    )
    naive_bayes.fit(X_treino, y_treino)

    # 2. Arvore de Decisao + GridSearchCV OBRIGATORIO.
    arvore = criar_pipeline(DecisionTreeClassifier(random_state=42))
    parametros = {
        "classificador__criterion": ["gini", "entropy"],
        "classificador__max_depth": [3, 5, 7, None],
        "classificador__min_samples_leaf": [1, 5],
    }
    busca = GridSearchCV(
        arvore, param_grid=parametros, cv=cv,
        scoring="f1_macro", n_jobs=-1, refit=True
    )

    # 3. cross_val_score sobre a PROPRIA busca: validacao cruzada aninhada.
    # A escolha de hiperparametros acontece apenas nas dobras internas.
    scores_arvore = cross_val_score(
        busca, X_treino, y_treino, cv=cv, scoring="accuracy"
    )
    busca.fit(X_treino, y_treino)
    melhor_arvore = busca.best_estimator_

    # 4. Comparar desempenho da validacao cruzada e do teste reservado.
    resultados = []
    for nome, modelo, scores in [
        ("Naive Bayes", naive_bayes, scores_nb),
        ("Arvore de Decisao", melhor_arvore, scores_arvore),
    ]:
        previsoes = modelo.predict(X_teste)
        acuracia = accuracy_score(y_teste, previsoes)
        f1 = f1_score(y_teste, previsoes, average="macro")
        print("\n===", nome, "===")
        print("Acuracias nas 5 dobras:", np.round(scores, 4))
        print("CV media +/- desvio:", round(scores.mean(), 4),
              "+/-", round(scores.std(), 4))
        print("Acuracia no teste:", round(acuracia, 4))
        print("F1-macro no teste:", round(f1, 4))
        print("Relatorio:\n", classification_report(
            y_teste, previsoes, labels=CLASSES, zero_division=0
        ))
        matriz = confusion_matrix(y_teste, previsoes, labels=CLASSES)
        print("Matriz de confusao (Baixo, Medio, Alto):\n", matriz)
        resultados.append({
            "algoritmo": nome, "cv_media": scores.mean(),
            "cv_desvio": scores.std(), "acuracia_teste": acuracia,
            "f1_macro_teste": f1,
        })
    print("\nMelhores hiperparametros da Arvore:", busca.best_params_)

    RESULTADOS.mkdir(parents=True, exist_ok=True)
    pd.DataFrame(resultados).round(4).to_csv(
        RESULTADOS / "atividade4_comparativo.csv", index=False
    )
    print("\nResumo salvo em resultados/atividade4_comparativo.csv")


if __name__ == "__main__":
    main()
```